

SAKARYA ÜNİVERSİTESİ



SİSTEM PROGRAMLAMA KISA SINAV PROJESİ C PROGRAMLAMA VE LINUX SİNYALLERİ

Adı Soyadı	Numara	Şube
Emirhan Fırat	G231210380	1-A
Muhammed Güntay	B231210300	1-A

ÖĞRETİM GÖREVLİSİ
Dr.Öğr.Üyesi HÜSEYİN ESKİ

Github link: https://github.com/EmirhanFirat/Sistem_Programlama_ikili_proje

ÖZET

Bu raporda Sistem Programlama dersi kapsamında verilen iki kısa sınav ödevinin çözüm süreci açıklanmıştır. Birinci soruda malloc ile dinamik olarak ayrılmış tek boyutlu bellek bloğu üzerinde N x N matris işlemleri yapılmış ve yalnızca işaretçi aritmetiği kullanılarak ana diyagonal ile ikincil diyagonal toplamı hesaplanmıştır. İkinci soruda ise Linux süreçleri ve sinyal mekanizması kullanılarak fork(), signal(), alarm() ve kill() fonksiyonları ile çocuk sürecin durdurulması, devam ettirilmesi ve sonlandırılması gösterilmiştir.

Çalışmaların amacı sadece çalışan kod üretmek değildir. Aynı zamanda C dilinde bellek yönetimini, pointer mantığını, Linux süreç modelini ve sinyal işleme kavramlarını uygulamalı biçimde anlamaktır. Raporun ekran görüntüsü alanları bilinçli olarak boş bırakılmıştır; program çıktıları daha sonra bu alanlara eklenecektir.

GİRİŞ

Sistem programlama, uygulama seviyesindeki yazılımların işletim sistemiyle nasıl etkileşim kurduğunu anlamayı gerektirir. Bu ödevde ilk olarak C dilinde dinamik bellek yönetimi ve işaretçi aritmetiği üzerinde durulmuştur. İkinci bölümde ise Linux işletim sisteminde process oluşturma ve processlere sinyal gönderme konuları ele alınmıştır.

Bu iki problem birlikte değerlendirildiğinde, C dilinin düşük seviyeli bellek erişimi ve Linux'un süreç yönetimi mekanizmaları hakkında temel ancak önemli bir pratik kazandırmaktadır.

ÖDEVİN AMACI VE KAPSAMI

Amaç

Bu ödevin amacı, C programlama dili kullanılarak dinamik bellek tahsisi, işaretçi aritmetiği, süreç oluşturma ve sinyal işleme kavramlarının uygulanmasıdır. Birinci soru bellek ve pointer kullanımına, ikinci soru ise Linux sinyalleri ve parent-child süreç ilişkisine odaklanmaktadır.

Kapsam ve Sınırlar

- Her soru ayrı bir C kaynak dosyası olarak hazırlanmıştır.
- Soru 1'de iki boyutlu dizi indeksleme kullanılmamış, matris tek boyutlu bellek bloğunda tutulmuştur.
- Soru 2 Linux ortamında test edilmiştir; fork(), signal(), alarm() ve kill() fonksiyonları kullanılmıştır.
- Ekran görüntüleri için raporda ayrı alanlar bırakılmıştır.
- Kodlarda algoritmanın işleyişini açıklayan yorum satırları kullanılmıştır.

KULLANILAN ARAÇLAR

Araç / Teknoloji	Kullanım Amacı
C Programlama Dili	Kaynak kodların geliştirilmesi.
GCC	C dosyalarının Linux üzerinde derlenmesi.
Linux Terminal	Programların çalıştırılması ve test çıktılarının alınması.
malloc / free	Dinamik bellek tahsisi ve bellek serbest bırakma.
fork / signal / alarm / kill	Süreç oluşturma ve sinyal yönetimi.

SORU 1 - DİNAMİK MATRİS VE İŞARETÇİ ARİTMETİĞİ

Problem Tanımı

Birinci soruda kullanıcıdan $N \times N$ boyutunda bir matris alınmaktadır. Matris malloc ile dinamik olarak ayrılan tek boyutlu bir bellek bloğunda saklanmaktadır. Program, ana diyagonal ve ikincil diyagonal üzerindeki elemanların toplamını hesaplamaktadır. Matris boyutu tek sayı olduğunda merkez eleman iki diyagonalin kesişiminde yer aldığı için yalnızca bir kez hesaba katılmaktadır.

Bellekte Matrisin Temsili

Matris iki boyutlu görünse de bellekte ardışık tek boyutlu bir alan olarak tutulmuştur. Bu nedenle i . satır ve j . sütundaki elemana erişmek için $i * \text{cols} + j$ formülü kullanılmıştır. Kodda $\text{mat}[i][j]$ yerine $*(\text{mat} + i * \text{cols} + j)$ ifadesi kullanılmıştır.

Algoritma Adımları

- ☐ Kullanıcıdan matris boyutu N alınır.
- ☐ $N \times N$ adet int için malloc ile bellek ayrılır.
- ☐ Matris elemanları kullanıcıdan pointer aritmetiği ile okunur.
- ☐ special_sum fonksiyonu çağrılır.
- ☐ Fonksiyon ana ve ikincil diyagonal şartlarını kontrol eder.
- ☐ Toplam değer ekrana yazdırılır ve ayrılan bellek free ile serbest bırakılır.

Kaynak Kod

```
#include <stdio.h>
#include <stdlib.h>

/*
special_sum fonksiyonu:
Dinamik olarak ayrılmış tek boyutlu matris üzerinde
ana diyagonal ve yan diyagonal elemanların toplamını hesaplar.
*/
int special_sum(int *mat, int rows, int cols)
{
    int sum = 0;

    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            if (i == j || i + j == cols - 1)
            {
                sum += *(mat + i * cols + j);
            }
        }
    }
    return sum;
}

int main()
{
    int n;
    int *mat;
    printf("Matris boyutunu giriniz (N): ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Hata: Matris boyutu pozitif bir değer olmalıdır.\n");
        return 1;
    }

    mat = (int *)malloc(n * n * sizeof(int));
```

```
if (mat == NULL)
{
    printf("Hata: Bellek tahsis edilemedi.\n");
    return 1;
}

printf("%d x %d matris elemanlarini giriniz:\n", n, n);

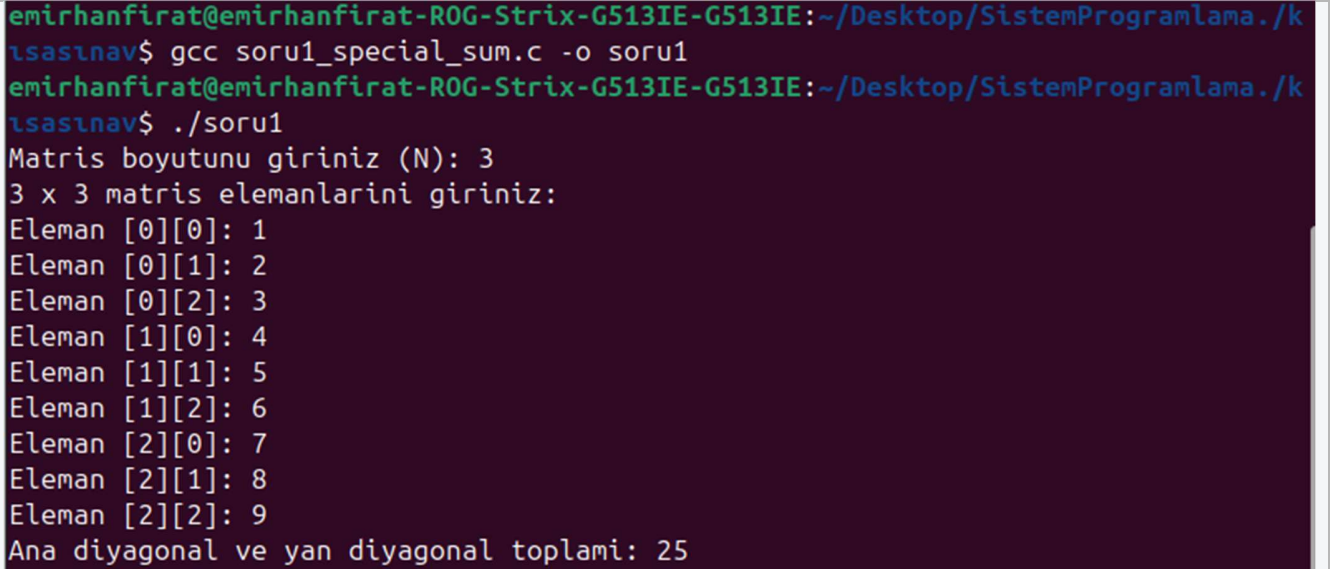
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        printf("Eleman [%d][%d]: ", i, j);
        scanf("%d", (mat + i * n + j));
    }
}

int result = special_sum(mat, n, n);
printf("Ana diyagonal ve yan diyagonal toplami: %d\n", result);

free(mat);
return 0;
}
```

Çalışma Çıktısı

Aşağıdaki alana Soru 1 programının Linux terminal çıktısına ait ekran görüntüsü eklenecektir.



```
emirhanfirat@emirhanfirat-ROG-Strix-G513IE-G513IE:~/Desktop/SistemProgramlama./k
ısasınav$ gcc soru1_special_sum.c -o soru1
emirhanfirat@emirhanfirat-ROG-Strix-G513IE-G513IE:~/Desktop/SistemProgramlama./k
ısasınav$ ./soru1
Matris boyutunu giriniz (N): 3
3 x 3 matris elemanlarini giriniz:
Eleman [0][0]: 1
Eleman [0][1]: 2
Eleman [0][2]: 3
Eleman [1][0]: 4
Eleman [1][1]: 5
Eleman [1][2]: 6
Eleman [2][0]: 7
Eleman [2][1]: 8
Eleman [2][2]: 9
Ana diyagonal ve yan diyagonal toplami: 25
```

Test Yorumu

Program 3 x 3 matris için 4, 5, 2, 1 değerleriyle test edildiğinde ana diyagonal toplamı 1 + 5 + 9, ikincil diyagonal toplamı ise 3 + 5 + 7 olmaktadır. 5 değeri iki defa sayıldığından toplamdan çıkarırız. Bu nedenle beklenen sonuç 25'dur. Program çıktısı bu sonucu verdiği için Soru 1 çözümü doğru çalışmıştır.

SORU 2 - LINUX SİNYAL İŞLEME

Problem Tanımı

İkinci soruda fork() ile bir çocuk süreç oluşturulmuştur. Çocuk süreç sonsuz döngü içinde her saniye sayaç yazdırmaktadır. Ana süreç ise alarm(3) mekanizması ile belirli aralıklarla SIGALRM sinyali almakta, her alarmda çocuk süreci önce SIGSTOP ile durdurmakta, iki saniye bekledikten sonra SIGCONT ile devam ettirmektedir. Yaklaşık çalışma süreci sonunda çocuk sürece SIGINT gönderilmekte, çocuk bu sinyali yakalayıp mesaj verdikten sonra ana süreç tarafından sonlandırılmaktadır.

Linux Süreç ve Sinyal Mantığı

Linux'ta çalışan her program bir süreç olarak ele alınır. fork() fonksiyonu mevcut sürecin bir kopyasını oluşturarak parent ve child olmak üzere iki ayrı akış meydana getirir. Sinyaller ise süreçlere gönderilen özel bildirimlerdir. Bu çalışmada SIGALRM zamanlama amacıyla, SIGSTOP durdurma amacıyla, SIGCONT devam ettirme amacıyla ve SIGINT kesme sinyalini göstermek amacıyla kullanılmıştır.

SIGSTOP sinyali yakalanamaz ve engellenemez. Bu nedenle çocuk süreçte SIGSTOP için bir signal handler yazılmamıştır. Buna karşılık SIGINT ve SIGCONT sinyalleri çocuk süreç tarafından yakalanarak ekrana mesaj yazdırılmıştır.

Algoritma Adımları

- ☐ Program fork() çağrısı ile parent ve child süreçlere ayrılır.
- ☐ Child süreç SIGINT ve SIGCONT sinyalleri için handler fonksiyonları tanımlar.
- ☐ Child süreç sonsuz döngüde sayaç yazdırır.
- ☐ Parent süreç SIGALRM için handler fonksiyonu tanımlar ve alarm(3) çağırır.
- ☐ Her alarmda child sürece SIGSTOP gönderilir, iki saniye beklenir ve SIGCONT gönderilir.
- ☐ İki alarm döngüsünden sonra parent süreç child sürece SIGINT gönderir.
- ☐ Child süreç SIGINT mesajını yazdırır, ardından parent süreç child süreci SIGTERM ile sonlandırır.

Kaynak Kod

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>

pid_t child_pid;
int alarm_count = 0;

void handle_sigint(int sig)
{
    printf("Çocuk: SIGINT alındı ancak devam ediliyor...\n");
}

void handle_sigcont(int sig)
{
    printf("Çocuk: İşlem yeniden başlatıldı\n");
}

void handle_sigalrm(int sig)
{
    alarm_count++;

    printf("Ebeveyn: Çocuk durduruluyor...\n");
    kill(child_pid, SIGSTOP);

    sleep(2);
}
```

```
printf("Ebeveyn: Çocuk devam ediyor...\n");
kill(child_pid, SIGCONT);

if (alarm_count < 2)
{
    alarm(3);
}
}

int main()
{
    child_pid = fork();

    if (child_pid < 0)
    {
        printf("Hata: fork işlemi başarısız oldu.\n");
        return 1;
    }

    if (child_pid == 0)
    {
        int counter = 0;

        signal(SIGINT, handle_sigint);
        signal(SIGCONT, handle_sigcont);

        while (1)
        {
            printf("Çocuk sayacı: %d\n", counter);
            counter++;
            sleep(1);
        }
    }
    else
    {
        signal(SIGALRM, handle_sigalrm);
        alarm(3);

        while (alarm_count < 2)
        {
            pause();
        }

        printf("Ebeveyn: SIGINT gönderiliyor...\n");
        kill(child_pid, SIGINT);

        sleep(1);
        kill(child_pid, SIGTERM);
        wait(NULL);

        printf("Ebeveyn: Çocuk sonlandırıldı.\n");
    }

    return 0;
}
```

Çalışma Çıktısı

Aşağıdaki alana Soru 2 programının Linux terminal çıktısına ait ekran görüntüsü eklenecektir. Ekran çıktısında sayaç değerleri, çocuk sürecin durdurulması, devam ettirilmesi, SIGINT mesajı ve sonlandırma mesajı görülmelidir.

```
emirhanfirat@emirhanfirat-ROG-Strix-G513IE-G513IE:~/Desktop/SistemProgramlama./k  
ısasınav$ gcc soru2_signal_handling.c -o soru2  
emirhanfirat@emirhanfirat-ROG-Strix-G513IE-G513IE:~/Desktop/SistemProgramlama./k  
ısasınav$ ./soru2  
Çocuk sayacı: 0  
Çocuk sayacı: 1  
Çocuk sayacı: 2  
Ebeveyn: Çocuk durduruluyor...  
Ebeveyn: Çocuk devam ediyor...  
Çocuk: İşlem yeniden başlatıldı  
Çocuk sayacı: 3  
Çocuk sayacı: 4  
Çocuk sayacı: 5  
Ebeveyn: Çocuk durduruluyor...  
Ebeveyn: Çocuk devam ediyor...  
Ebeveyn: SIGINT gönderiliyor...  
Çocuk: İşlem yeniden başlatıldı  
Çocuk: SIGINT alındı ancak devam ediliyor...  
Çocuk sayacı: 6  
Ebeveyn: Çocuk sonlandırıldı.
```

Test Yorumu

Program çalıştırıldığında çocuk sürecin sayaç yazdırdığı, ana sürecin çocuğu iki kez durdurup devam ettirdiği, SIGINT sinyalinin çocuk tarafından yakalanarak mesaj üretildiği ve son olarak çocuk sürecin sonlandırıldığı görülmüştür. Terminaldeki mesaj sıralamasında küçük farklılıklar oluşabilir. Bu durum Linux zamanlayıcısının parent ve child süreçleri çalışma zamanında farklı sıralarda yürütmesinden kaynaklanır ve beklenen bir durumdur.

KARŞILAŞILAN DURUMLAR VE ÇÖZÜMLER

Durum	Nasıl Ortaya Çıktı?	Çözüm / Yorum
Pointer indeksleme yasağı	Matris iki boyutlu gibi düşünülmesine rağmen [][] kullanımı istenmedi.	Eleman erişimi için $*(mat + i * cols + j)$ formülü kullanıldı.
Merkez elemanın iki kez sayılması	Tek boyutlu kare matrislerde orta eleman iki diyagonal üzerinde de bulunabilir.	if koşulunda OR kullanılarak elemanın yalnızca bir kez toplanması sağlandı.
sleep(10) erken kesildi	SIGALRM sinyali parent sürecin sleep çağrısını böldü.	Alarm sayacı ve pause() kullanılarak iki alarm tamamlanana kadar bekleme yapıldı.
SIGSTOP yakalanamaz	SIGSTOP için handler yazılamayacağı görüldü.	SIGSTOP doğrudan parent tarafından gönderildi, child tarafında handler tanımlanmadı.
Çıktı sırası değişebiliyor	Parent ve child süreçlerin ekrana yazma sırası her çalıştırmada aynı olmayabilir.	Bu durum Linux scheduler davranışı olarak yorumlandı; ana mesajların üretilmesi yeterli kabul edildi.

SONUÇ

Bu çalışma sonucunda C dilinde dinamik bellek tahsisi ve işaretçi aritmetiği kullanılarak matris üzerinde işlem yapılmıştır. Ayrıca Linux ortamında süreç oluşturma ve sinyal gönderme mekanizmaları uygulanmıştır. Birinci programda bellek yönetiminin ve pointer aritmetiğinin önemi görülmüş, ikinci programda ise parent-child süreç ilişkisi, sinyal yakalama ve sinyal gönderme kavramları pratik olarak test edilmiştir.

Ödev kapsamında geliştirilen iki program da beklenen çıktıları üretmiştir. Ekran görüntüleri rapora eklendikten sonra çalışma teslim formatına uygun hale gelecektir.

EKLER

- ☐ Ek A - Soru 1 Linux terminal ekran görüntüsü
- ☐ Ek B - Soru 2 Linux terminal ekran görüntüsü
- ☐ Ek C - Kaynak kod dosyaları: soru1_special_sum.c ve soru2_signal_handling.c